Grafika komputerowa i wizualizacja

Transformacje obrazów graficznych

Imię i nazwisko: Maksymilian Chojniak
Numer albumu: 140636
Kierunek: Sztuczna Inteligencja, semestr 4
Data: 2026-04-21

1. Wstęp

Współczesne przetwarzanie obrazów cyfrowych w środowisku programistycznym C++ przy użyciu biblioteki GLFW opiera się na matematycznych operacjach wykonywanych bezpośrednio na strukturze pikseli. Kluczowym aspektem tych działań jest podział transformacji na metody bezkontekstowe, splotowe oraz statystyczne. Przekształcenia bezkontekstowe modyfikują kolor każdego piksela niezależnie, bazując jedynie na jego pierwotnych składowych RGB, co wykorzystuje się m.in. przy filtrach kanałowych czy zmianie jasności. Z kolei filtry splotowe, będące metodami kontekstowymi, wyliczają nową wartość piksela na podstawie sumy ważonej jego otoczenia przy użyciu dedykowanych masek, co pozwala na wyodrębnianie krawędzi lub rozmywanie obrazu. Ostatnią grupę stanowią filtry statystyczne, które zamiast operacji liniowych wykorzystują operacje statystyczne.

2. Oryginalny obraz

Oryginalny obraz jest wczytywany poprzez:

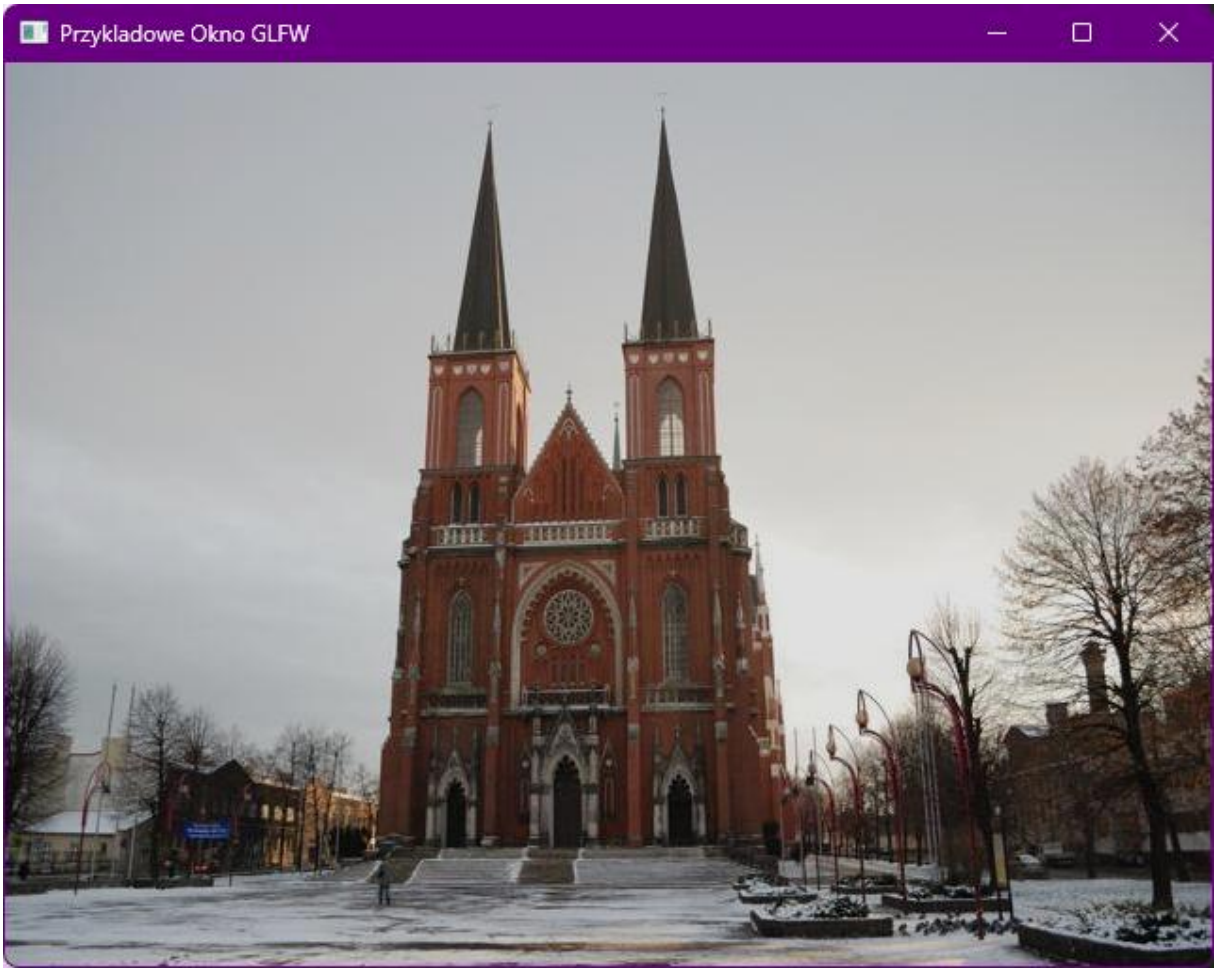
```
ifstream plik("C:path/zdjecie.txt");
plik >> lw >> lk;
cout << "wiersze=" << w << " kolumny=" << k << endl;
for (int i = 0; i < w; ++i)
{
    for (int j = 0; j < lk; ++j)
    {
        plik >> Rs[i][j];
        plik >> Gs[i][j];
        plik >> Bs[i][j];

        Rc[i][j] = Rs[i][j];
        Gc[i][j] = Gs[i][j];
        Bc[i][j] = Bs[i][j];
    }
}
plik.close();
```

oraz wyświetlany za pomocą:

```
glPushMatrix();
glTranslated(0, 0, -6);
glBegin(GL_POINTS);
for (int i = 0; i < w; ++i)
{
    for (int j = 0; j < lk; ++j)
    {
        glPointSize(ps);
        glColor3d((Rc[i][j] + brightness) / 255.0, (Gc[i][j] + brightness) / 255.0,
(Bc[i][j] + brightness) / 255.0);
        glVertex3f(j, i, 0);
    }
}
glEnd();
glPopMatrix();
```

Oryginalny obraz:



Zrzut Ekranu 1 - Oryginalny Bazowy

3. Przekształcenia bezkontekstowe

• Filtr rozjaśniający

Operacja ta polega na dodaniu (rozjaśnianie) stałej wartości od każdego kanału koloru piksela.

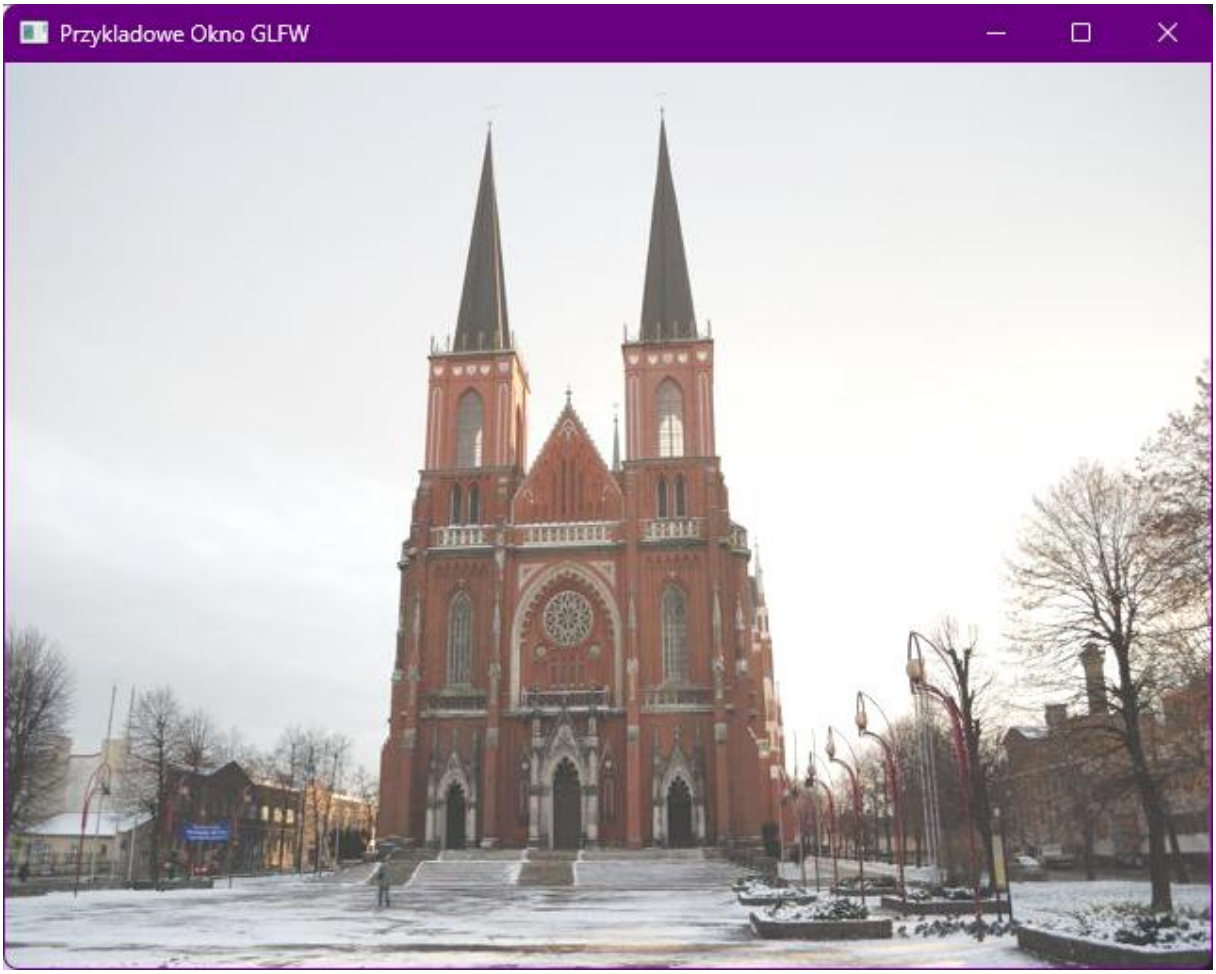
Jest to wykonane poprzez zwiększenie stałej ‘brightness’:

```
case GLFW_KEY_UP:
    brightness += brightness_step;
    break;
```

oraz dodanie do koloru bazowego:

```
glColor3d((Rc[i][j] + brightness) / 255.0, (Gc[i][j] + brightness) / 255.0, (Bc[i][j]
+ brightness) / 255.0);
```

Obraz o wykonaniu rozjaśnienia:



Zrzut Ekranu 2 - Filtr Rozjaśniający

- ## Filtr przyciemniający

Operacja ta polega na odjęciu (przyciemnianie) stałej wartości od każdego kanału koloru piksela.

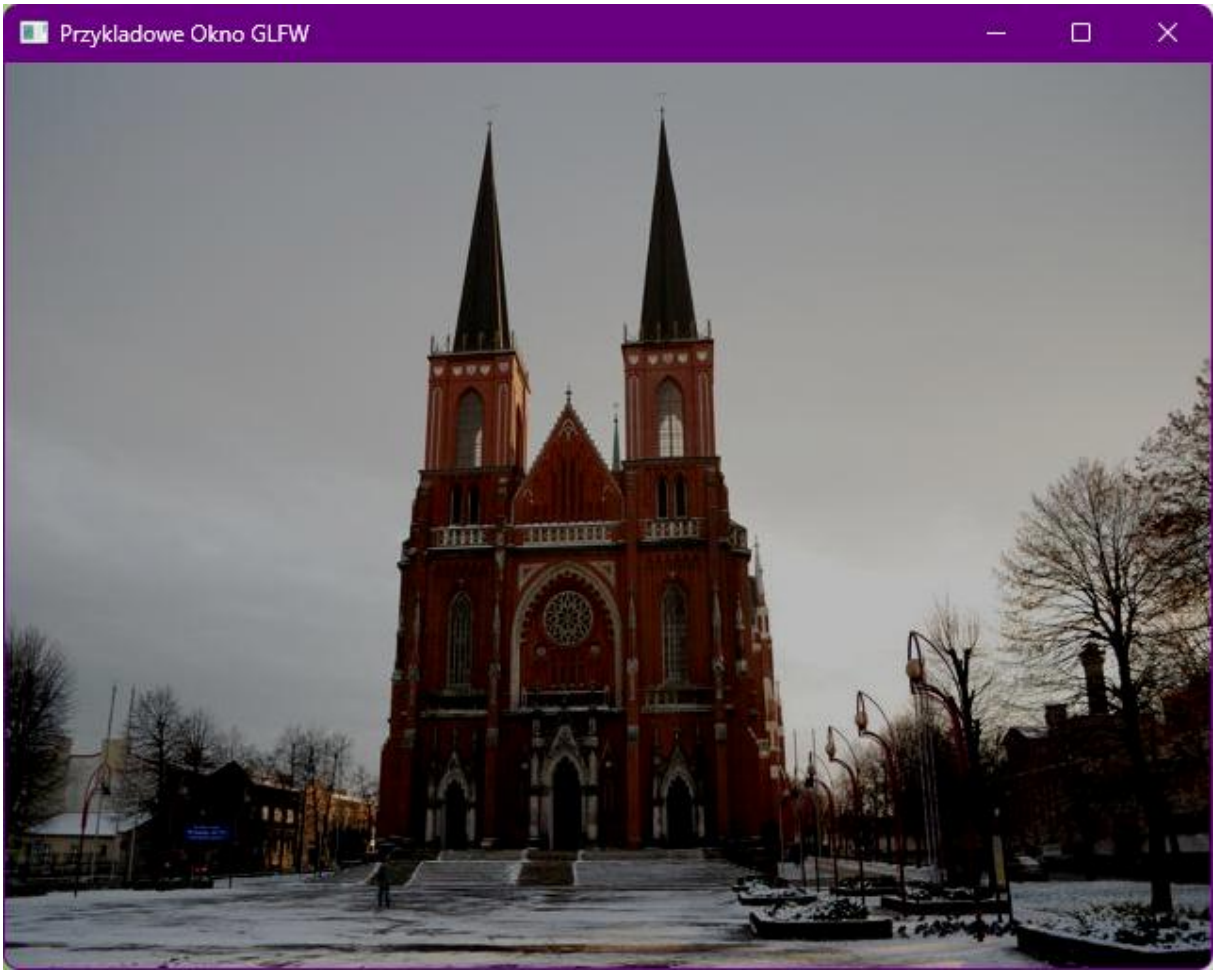
Jest to wykonane poprzez zwiększenie stałej ‘brightness’:

```
case GLFW_KEY_DOWN:
    brightness -= brightness_step;
    break;
```

oraz dodanie do koloru bazowego:

```
glColor3d((Rc[i][j] + brightness) / 255.0, (Gc[i][j] + brightness) / 255.0, (Bc[i][j]
+ brightness) / 255.0);
```

Obraz o wykonaniu przyciemnienia:



Zrzut Ekranu 3 - Filtr Przyciemniający

- ## Filtr kanałowy (RGB)

Filtry te polegają na izolacji konkretnej składowej barwnej obrazu poprzez wyzerowanie pozostałych kanałów.

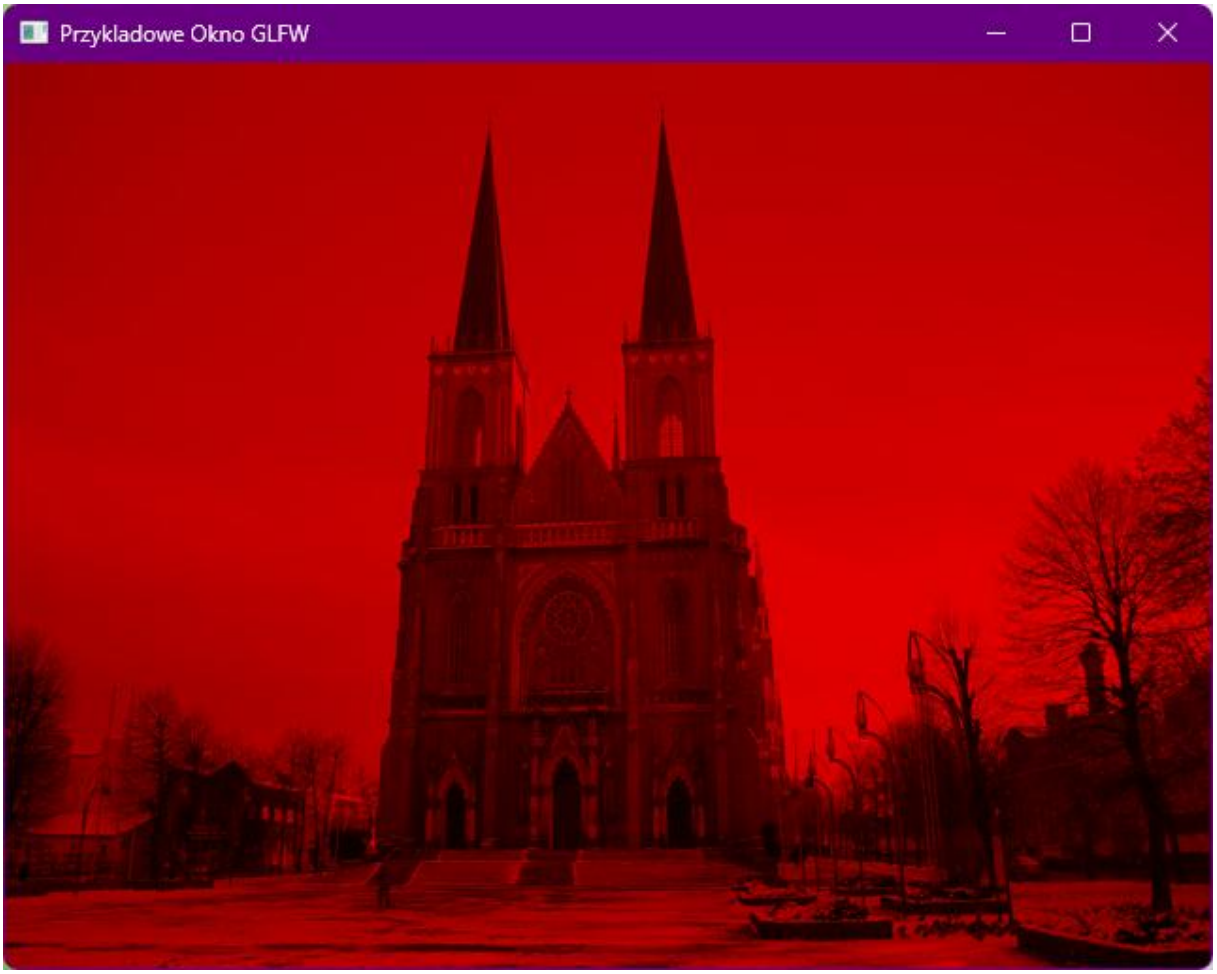
Kanał czerwony

Dla kanału czerwonego przyjmujemy: R=R, G=0, B=0.

Jest to wykonane poprzez:

```
case GLFW_KEY_R:
    for (int i = 0; i < w; ++i)
    {
        for (int j = 0; j < lk; ++j)
        {
            Rc[i][j] = Rs[i][j];
            Gc[i][j] = 0;
            Bc[i][j] = 0;
        }
    }
    break;
```

Obraz po zastosowaniu filtru dla kanału czerwonego:



Zrzut Ekranu 4 - Filtr Kanału Czerwonego

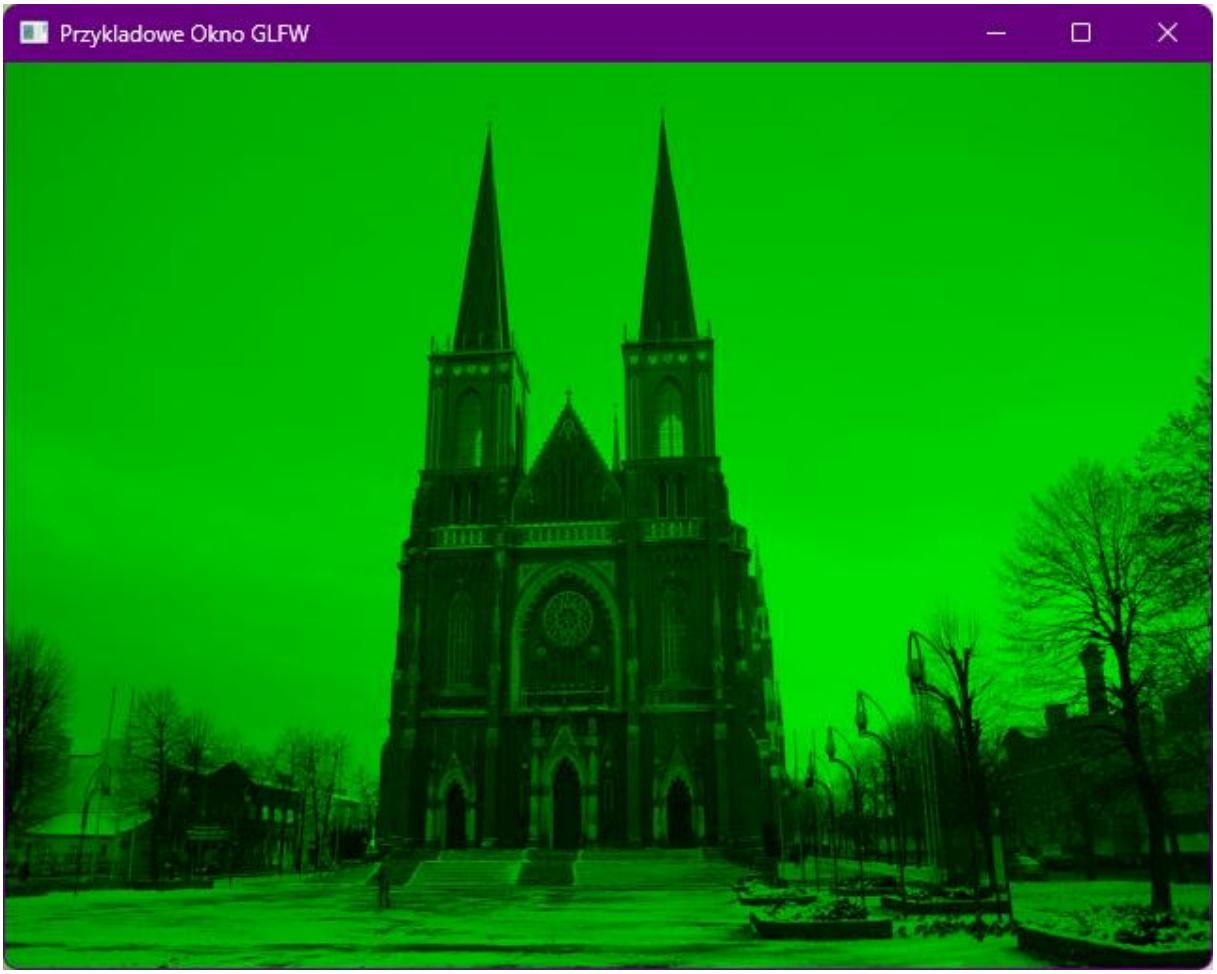
○ Kanał Zielony

Dla kanału zielonego przyjmujemy: R=0, G=G, B=0.

Jest to wykonane poprzez:

```
case GLFW_KEY_T:
    for (int i = 0; i < w; ++i)
    {
        for (int j = 0; j < lk; ++j)
        {
            Rc[i][j] = 0;
            Gc[i][j] = Gs[i][j];
            Bc[i][j] = 0;
        }
    }
    break;
```

Obraz po zastosowaniu filtru dla kanału zielongo:



Zrzut Ekranu 5 - Filtr Kanału Zielonego

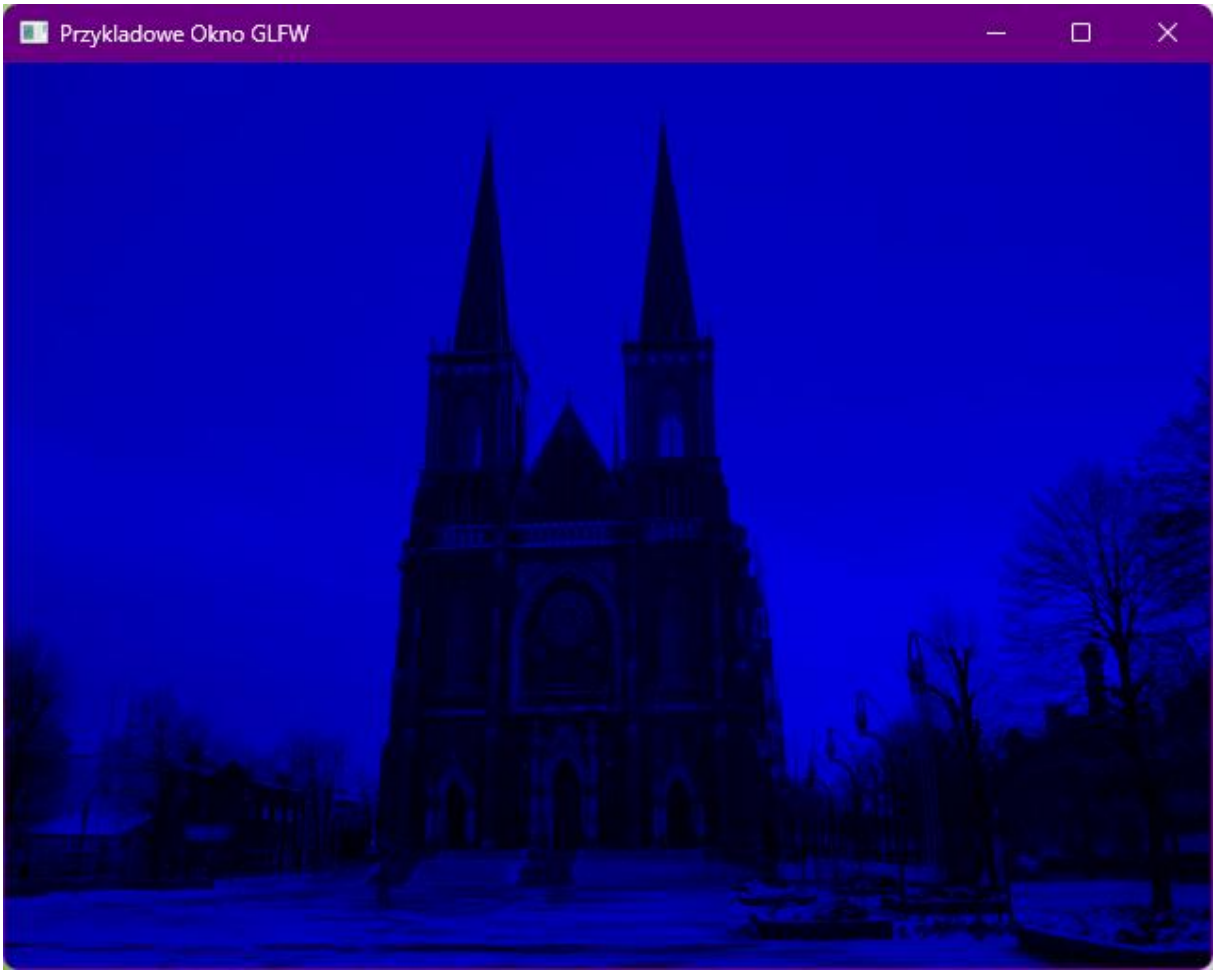
○ Kanał Niebieski

Dla kanału niebieskiego przyjmujemy: R=0, G=0, B=B.

Jest to wykonane poprzez:

```
case GLFW_KEY_Y:
    for (int i = 0; i < w; ++i)
    {
        for (int j = 0; j < lk; ++j)
        {
            Rc[i][j] = 0;
            Gc[i][j] = 0;
            Bc[i][j] = Bs[i][j];
        }
    }
    break;
```

Obraz po zastosowaniu filtru dla kanału niebieskiego:



Zrzut Ekranu 6 - Filtr Kanału Niebieskiego

4. Przekształcenia kontekstowe

• Filtry splotowe

Są to liniowe operacje kontekstowe polegające na przemieszczaniu nad obrazem maski o określonych wagach, gdzie nowa wartość piksela jest obliczana jako suma iloczynów sąsiednich pikseli i odpowiadających im współczynników maski. W tym przypadku zostały użyte maski 3x3.

Jest to wykonane poprzez wywołanie funkcji:

```
const int kernelSize = 3;

void zastosuj_maske(int m[kernelSize][kernelSize]) {
    int sum = 0;
    for (int s = 0; s < kernelSize; ++s)
    {
        for (int d = 0; d < kernelSize; ++d)
        {
            sum += m[s][d];
        }
    }
    if (sum == 0)
        sum = 1;

    for (int i = 1; i < w - 1; ++i)
    {
        for (int j = 1; j < lk - 1; ++j)
        {
            int pomR = 0;
            int pomG = 0;
            int pomB = 0;

            for (int s = 0; s < kernelSize; ++s)
            {
                for (int d = 0; d < kernelSize; ++d)
                {
                    pomR = pomR + (m[s][d] * Rs[i - 1 + s][j - 1 + d]);
                    pomG = pomG + (m[s][d] * Gs[i - 1 + s][j - 1 + d]);
                    pomB = pomB + (m[s][d] * Bs[i - 1 + s][j - 1 + d]);
                }
            }
            Rc[i][j] = pomR / sum;
            Gc[i][j] = pomG / sum;
            Bc[i][j] = pomB / sum;
        }
    }
}
```

- ### Dolnoprzepustowy

Ma na celu wygładzenie obrazu i redukcję wysokich częstotliwości (szumów).

Jest to wykonane poprzez użycie maski:

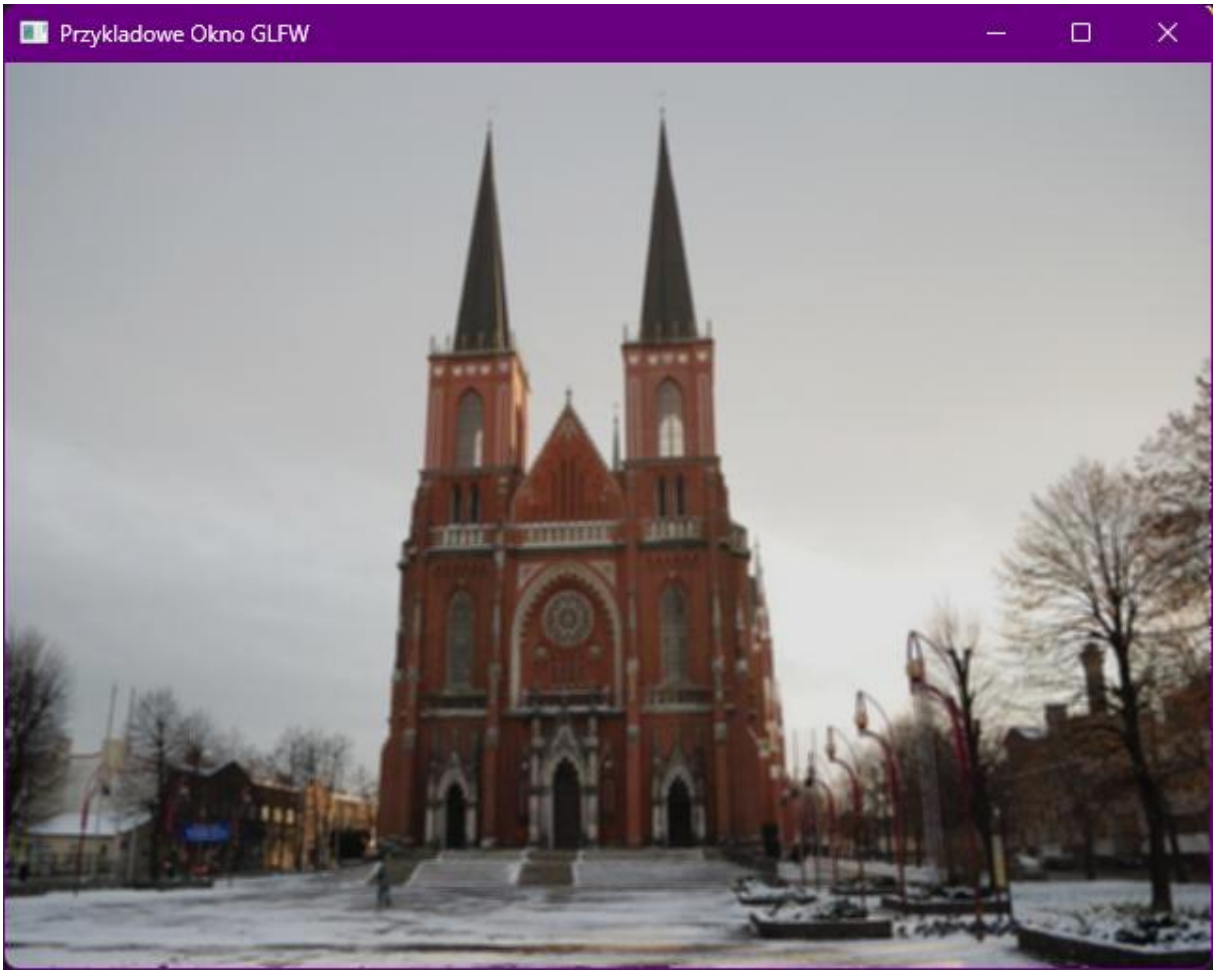
```
const int kernelSize = 3;

int Mdp[kernelSize][kernelSize] = {
    {1,1,1},
    {1,1,1},
    {1,1,1}
};
```

Oraz wywołanie metody:

```
case GLFW_KEY_D:
    zastosuj_maske(Mdp);
    break;
```

Obraz po zastosowaniu filtru:



Zrzut Ekranu 7 - Filtr Dolnoprzepustowy

○ Górnoprzepustowy

Wzmacnia różnice między pikselami, co powoduje wyostrzenie krawędzi i detali.

Jest to wykonane poprzez użycie maski:

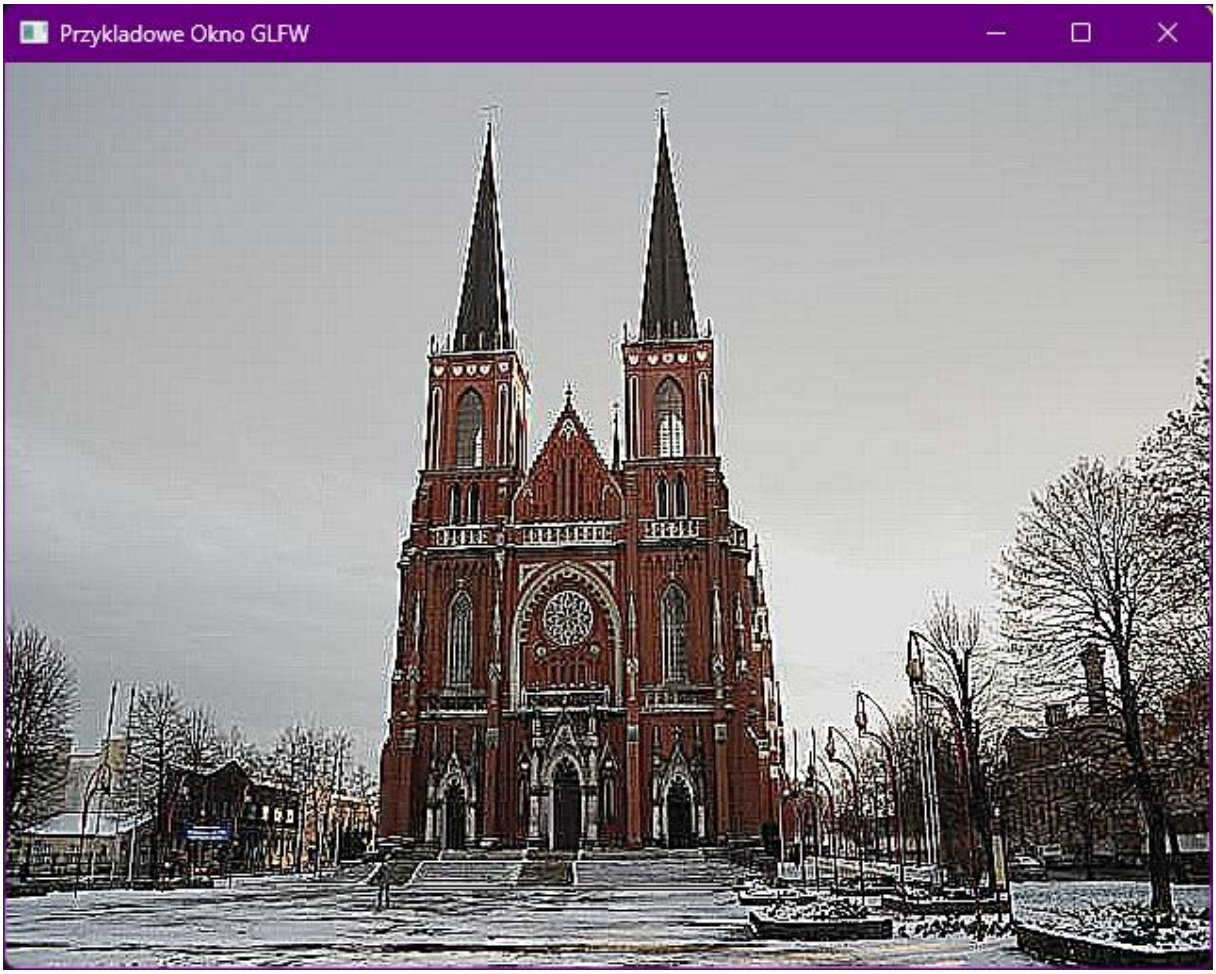
```
const int kernelSize = 3;

int Mgp[kernelSize][kernelSize] = {
    {-1,-1,-1},
    {-1, 9,-1},
    {-1,-1,-1}
};
```

Oraz wywołanie metody:

```
case GLFW_KEY_C:
    zastosuj_maske(Mgp);
    break;
```

Obraz po zastosowaniu filtru:



Zrzut Ekranu 8 - Filtr Górnoprzepustowy

○ Krawędziowy Laplace'a

Służy do detekcji krawędzi bez względu na ich kierunek.

Jest to wykonane poprzez użycie maski:

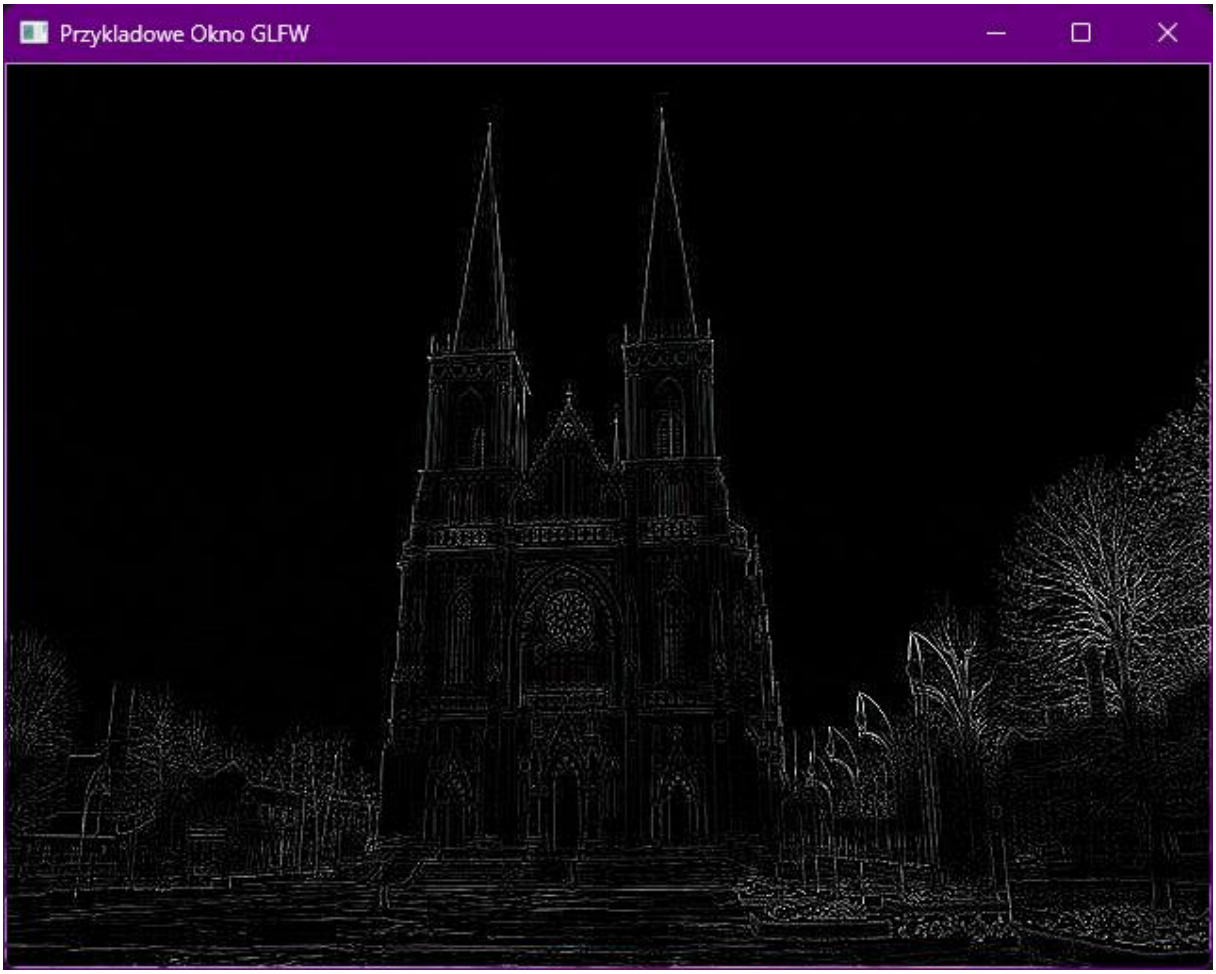
```
const int kernelSize = 3;

int Mkrawedz[kernelSize][kernelSize] = {
    {0,1,0},
    {1,-4,1},
    {0,1,0}
};
```

Oraz wywołanie metody:

```
case GLFW_KEY_X:
    zastosuj_maske(Mkrawedz);
    break;
```

Obraz po zastosowaniu filtru:



Zrzut Ekranu 9 - Filtr Krawędziowy Laplace'a

○ Emboss

Tworzy iluzję wypukłości obrazu poprzez badanie różnic jasności wzdłuż wybranego kierunku (zazwyczaj przekątnej).

Jest to wykonane poprzez użycie maski:

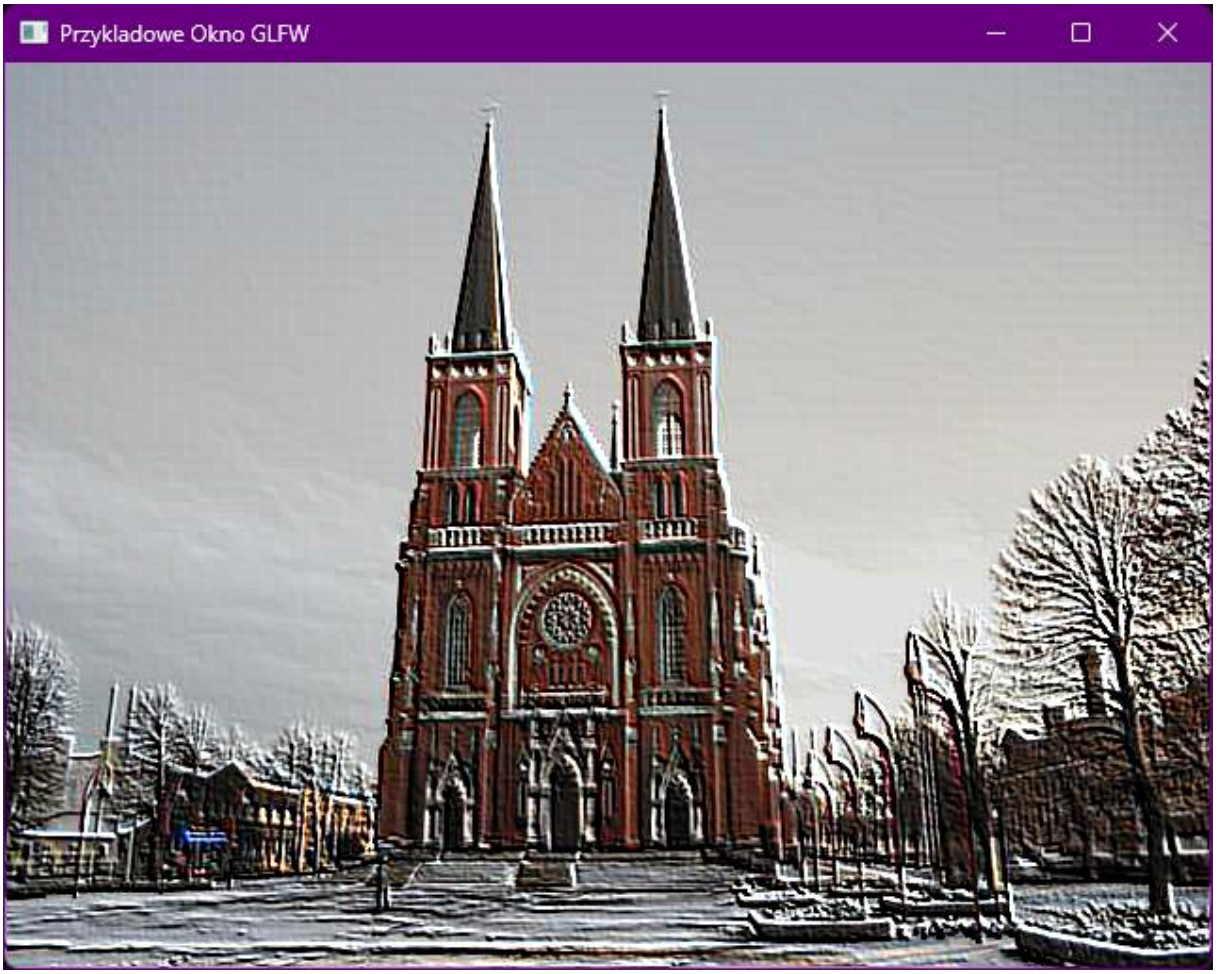
```
const int kernelSize = 3;

int Memboss[kernelSize][kernelSize] = {
    {-2,-1,0},
    {-1, 1,1},
    { 0, 1,2}
};
```

Oraz wywołanie metody:

```
case GLFW_KEY_Z:
    zastosuj_maske(Memboss);
    break;
```

Obraz po zastosowaniu filtru:



Zrzut Ekranu 10 - Filtr Emboss

- ## Filtry statystyczne

Filtry te działają inaczej niż sploty, ponieważ nie mnożą wartości przez maskę, lecz wybierają konkretną wartość z otoczenia piksela na podstawie porównania statystycznego np. min czy max.

Jest to wykonane poprzez wywołanie funkcji:

```
void zastosuj_filtr(int (*fun)(int, int), int init) {


    for (int i = 1; i < w - 1; ++i)
    {
        for (int j = 1; j < lk - 1; ++j)
        {
            int maxR = init;
            int maxG = init;
            int maxB = init;
            for (int s = 0; s < kernelSize; ++s)
            {
                for (int d = 0; d < kernelSize; ++d)
                {
                    maxR = fun(maxR, Rs[i - 1 + s][j - 1 + d]);
                    maxG = fun(maxG, Gs[i - 1 + s][j - 1 + d]);
                    maxB = fun(maxB, Bs[i - 1 + s][j - 1 + d]);
                }
            }
            Rc[i][j] = maxR;
            Gc[i][j] = maxG;
            Bc[i][j] = maxB;
        }
    }
}
```

- ### Min

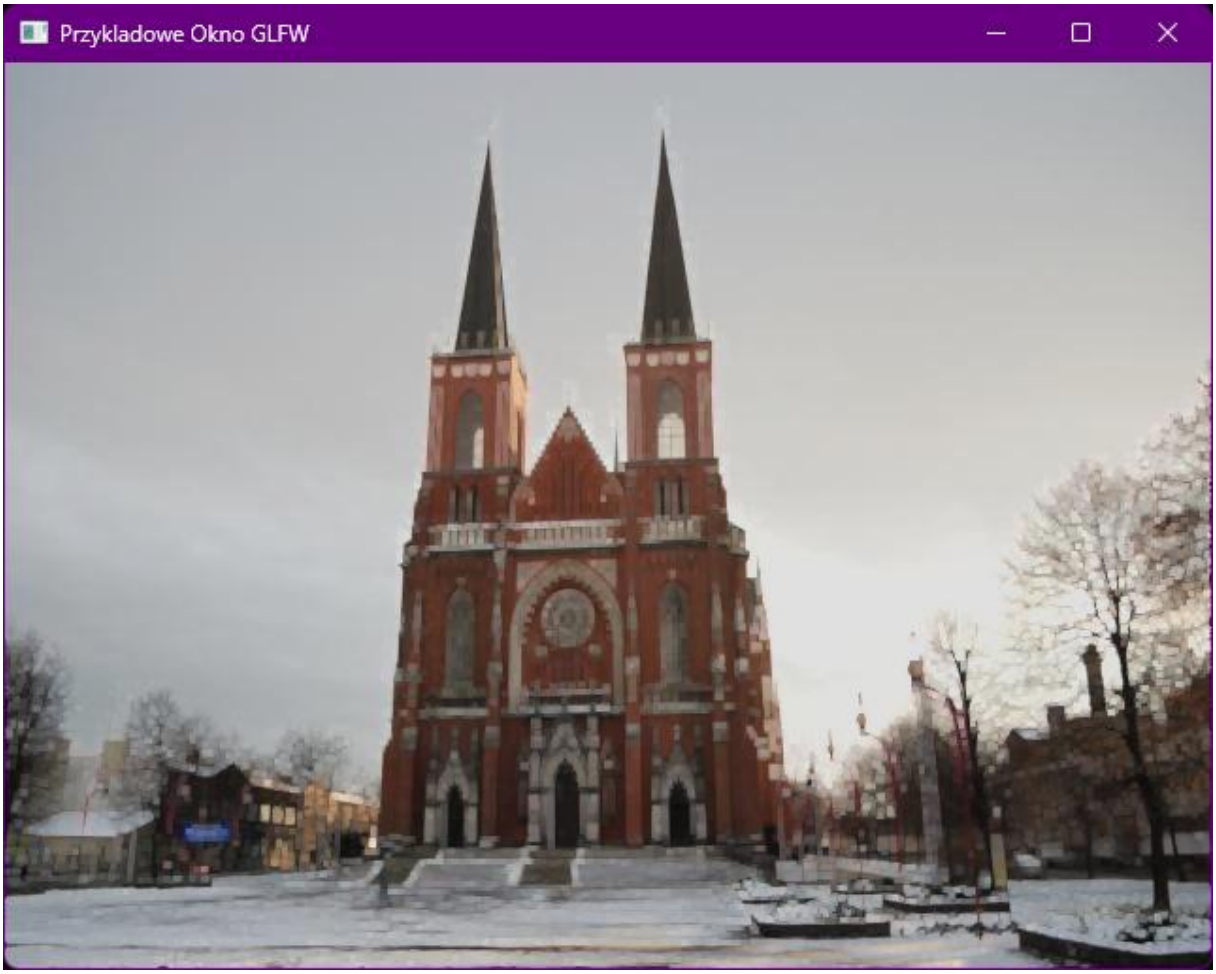
Jest to wykonane poprzez zdefiniowania funkcji min:

```
int min(int a, int b) {
        return a < b ? a : b;
}
```

oraz wywołanie metody:

```
case GLFW_KEY_G:
    zastosuj_filtr(min, INT32_MAX);
    break;
```

Obraz po zastosowaniu filtru:



Zrzut Ekranu 11 - Filtr Minimalny

○ Max

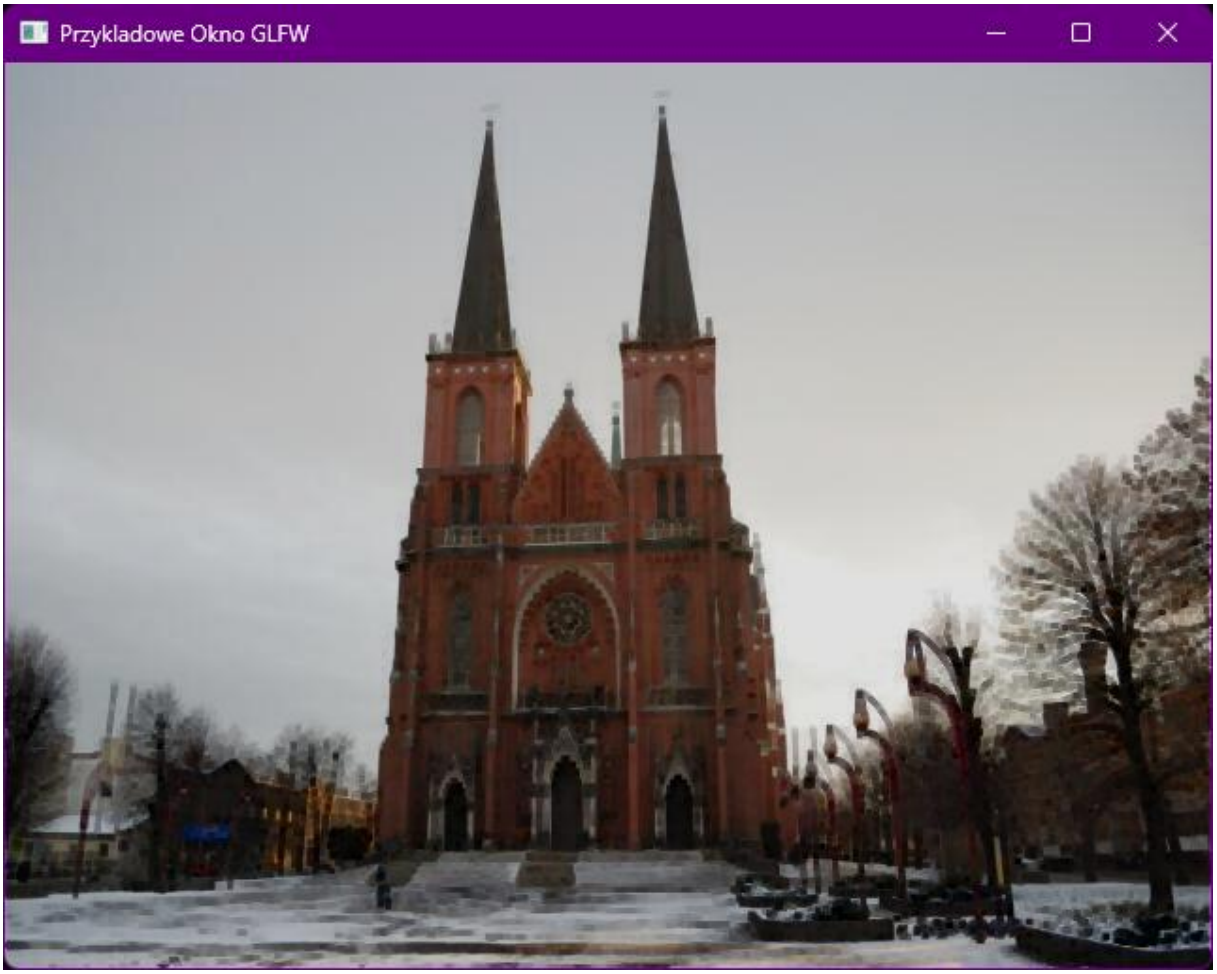
Jest to wykonane poprzez zdefiniowania funkcji min:

```
int max(int a, int b) {
    return a > b ? a : b;
}
```

oraz wywołanie metody:

```
case GLFW_KEY_F:
    zastosuj_filtr(max, INT32_MIN);
    break;
```

Obraz po zastosowaniu filtru:



Zrzut Ekranu 12 - Filtr Maksymalny

5. Wnioski

Przeprowadzone ćwiczenia potwierdziły, że wybór odpowiedniej metody transformacji zależy od pożądanego efektu końcowego. Przekształcenia bezkontekstowe najlepiej sprawdzają się przy szybkich korektach kolorystycznych, natomiast operacje kontekstowe (splotowe) są niezbędne do manipulacji ostrością i wykrywania krawędzi. Wykorzystanie filtrów statystycznych takich jak 'min' i 'max' pokazało z kolei, że nieliniowe podejście do sąsiedztwa pikseli pozwala na skuteczną zmianę grubości detali i kształtów obiektów na obrazie.

6. Kod Źródłowy:

Podaje link do github z kodem źródłowym, ponieważ cały kod się słabo formatuje:
https://github.com/MaksChojniak/OGL-sprawozdanie1